

NAME: AYESHA NOOR

R.NO: 2502054

DATA STRUCTURES LAB

HOME TASKS

HOME TASK 01:

CODE:

```
#include <iostream>
#include <string>
using namespace std;
```

```
// Node
```

```
struct Patient
```

```
{
```

```
    int id;
```

```
    string name;
```

```
    int age;
```

```
    Patient* next;
```

```
};
```

```
// Add patient at the end
```

```
void addPatient(Patient*& head)
```

```
{  
    Patient* newPatient = new Patient;  
  
    cout << "Enter Patient ID: ";  
    cin >> newPatient->id;  
  
    cout << "Enter Patient Name: ";  
    cin.ignore();  
    getline(cin, newPatient->name);  
  
    cout << "Enter Patient Age: ";  
    cin >> newPatient->age;  
  
    newPatient->next = NULL;  
  
    if (head == NULL)  
    {  
        head = newPatient;  
    }  
    else  
    {  
        Patient* temp = head;  
  
        while (temp->next != NULL)  
        {  
            temp = temp->next;  
        }  
    }  
}
```

```
temp->next = newPatient;

}

cout << "Patient added successfully.\n";

}

// Add emergency patient at beginning
void addEmergencyPatient(Patient*& head)
{
    Patient* newPatient = new Patient;

    cout << "Enter Patient ID: ";
    cin >> newPatient->id;

    cout << "Enter Patient Name: ";
    cin.ignore();
    getline(cin, newPatient->name);

    cout << "Enter Patient Age: ";
    cin >> newPatient->age;

    newPatient->next = head;
    head = newPatient;

    cout << "Emergency patient added at beginning.\n";
}
```

```
// Search patient

void searchPatient(Patient* head)
{
    int id;

    cout << "Enter Patient ID to search: ";
    cin >> id;

    Patient* temp = head;

    while (temp != NULL)
    {
        if (temp->id == id)
        {
            cout << "\nPatient Found!\n";
            cout << "ID: " << temp->id << endl;
            cout << "Name: " << temp->name << endl;
            cout << "Age: " << temp->age << endl;
            return;
        }

        temp = temp->next;
    }

    cout << "Patient not found.\n";
}
```

```
// Remove patient

void removePatient(Patient*& head)
{
    int id;

    cout << "Enter Patient ID to remove: ";
    cin >> id;

    if (head == NULL)
    {
        cout << "Patient not found.\n";
        return;
    }

    if (head->id == id)
    {
        Patient* temp = head;
        head = head->next;
        delete temp;

        cout << "Patient removed successfully.\n";
        return;
    }

    Patient* temp = head;
```

```

while (temp->next != NULL)
{
    if (temp->next->id == id)
    {
        Patient* deleteNode = temp->next;
        temp->next = temp->next->next;

        delete deleteNode;

        cout << "Patient removed successfully.\n";
        return;
    }

    temp = temp->next;
}

cout << "Patient not found.\n";
}

// Display patients
void displayPatients(Patient* head)
{
    if (head == NULL)
    {
        cout << "No patients waiting.\n";
        return;
    }
}

```

```
cout << "\nWaiting Patients:\n";
```

```
Patient* temp = head;
```

```
while (temp != NULL)
```

```
{
```

```
    cout << "ID: " << temp->id
```

```
        << ", Name: " << temp->name
```

```
        << ", Age: " << temp->age << endl;
```

```
    temp = temp->next;
```

```
}
```

```
}
```

```
// Main
```

```
int main()
```

```
{
```

```
    Patient* head = NULL;
```

```
    int choice;
```

```
    do
```

```
    {
```

```
        cout << "\n===== Hospital Emergency System =====\n";
```

```
        cout << "1. Add New Patient\n";
```

```
        cout << "2. Add Emergency Patient\n";
```

```
cout << "3. Search Patient\n";  
cout << "4. Remove Patient\n";  
cout << "5. Display All Patients\n";  
cout << "6. Exit\n";
```

```
cout << "Enter choice: ";  
cin >> choice;
```

```
switch (choice)  
{  
    case 1:  
        addPatient(head);  
        break;  
  
    case 2:  
        addEmergencyPatient(head);  
        break;  
  
    case 3:  
        searchPatient(head);  
        break;  
  
    case 4:  
        removePatient(head);  
        break;  
  
    case 5:
```

```

        displayPatients(head);

        break;

    case 6:

        cout << "Program ended.\n";

        break;

    default:

        cout << "Invalid choice.\n";

    }

} while (choice != 6);

return 0;
}

```

OUTPUT:

```

Waiting Patients Queue:
[ID: 101 | Name: Ali Khan | Age: 45] -> [ID: 102 | Name: Sara Ahmed | Age: 29]

--- Adding Emergency Patient ---
Waiting Patients Queue:
[ID: 100 | Name: Zaid Shah | Age: 60] -> [ID: 101 | Name: Ali Khan | Age: 45] -> [ID

--- Searching Patients ---
Patient Found: ID: 102 | Name: Sara Ahmed | Age: 29
Patient with ID 999 does not exist.

--- Treating/Removing Patient ---
Patient ID 100 (Zaid Shah) treated and removed.
Waiting Patients Queue:
[ID: 101 | Name: Ali Khan | Age: 45] -> [ID: 102 | Name: Sara Ahmed | Age: 29]

```

HOME TASK 02:

CODE:

```
#include <iostream>

#include <string>

using namespace std;


// Node

struct Student
{
    int rollNo;

    string name;

    string attendance;

    Student* next;
};


// Add student

void addStudent(Student*& head)
{
    Student* newStudent = new Student;

    cout << "Enter Roll Number: ";

    cin >> newStudent->rollNo;

    cout << "Enter Student Name: ";

    cin.ignore();

    getline(cin, newStudent->name);
```

```
cout << "Enter Attendance Status (Present/Absent): ";
cin >> newStudent->attendance;

newStudent->next = NULL;

if (head == NULL)
{
    head = newStudent;
}
else
{
    Student* temp = head;

    while (temp->next != NULL)
    {
        temp = temp->next;
    }

    temp->next = newStudent;
}

cout << "Student added successfully.\n";
}

// Search student
void searchStudent(Student* head)
{

```

```
int rollNo;
```

```
cout << "Enter Roll Number to search: ";
```

```
cin >> rollNo;
```

```
Student* temp = head;
```

```
while (temp != NULL)
```

```
{
```

```
    if (temp->rollNo == rollNo)
```

```
    {
```

```
        cout << "\nStudent Found!\n";
```

```
        cout << "Roll Number: " << temp->rollNo << endl;
```

```
        cout << "Name: " << temp->name << endl;
```

```
        cout << "Attendance: " << temp->attendance << endl;
```

```
        return;
```

```
    }
```

```
    temp = temp->next;
```

```
}
```

```
cout << "Student not found.\n";
```

```
}
```

```
// Delete student
```

```
void deleteStudent(Student*& head)
```

```
{
```

```
int rollNo;
```

```
cout << "Enter Roll Number to delete: ";
```

```
cin >> rollNo;
```

```
if (head == NULL)
```

```
{
```

```
    cout << "Student not found.\n";
```

```
    return;
```

```
}
```

```
if (head->rollNo == rollNo)
```

```
{
```

```
    Student* temp = head;
```

```
    head = head->next;
```

```
    delete temp;
```

```
    cout << "Student deleted successfully.\n";
```

```
    return;
```

```
}
```

```
Student* temp = head;
```

```
while (temp->next != NULL)
```

```
{
```

```
    if (temp->next->rollNo == rollNo)
```

```

{
    Student* deleteNode = temp->next;

    temp->next = temp->next->next;

    delete deleteNode;

    cout << "Student deleted successfully.\n";
    return;
}

temp = temp->next;
}

cout << "Student not found.\n";
}

// Display all students
void displayStudents(Student* head)
{
    if (head == NULL)
    {
        cout << "Attendance list is empty.\n";
        return;
    }

    cout << "\nAttendance List:\n";

```

```
Student* temp = head;
```

```
while (temp != NULL)
```

```
{
```

```
    cout << "Roll No: " << temp->rollNo
```

```
        << ", Name: " << temp->name
```

```
        << ", Status: " << temp->attendance << endl;
```

```
    temp = temp->next;
```

```
}
```

```
}
```

```
// Count present students
```

```
void countPresent(Student* head)
```

```
{
```

```
    int count = 0;
```

```
    Student* temp = head;
```

```
    while (temp != NULL)
```

```
    {
```

```
        if (temp->attendance == "Present" ||
```

```
            temp->attendance == "present")
```

```
        {
```

```
            count++;
```

```
        }
```

```

        temp = temp->next;
    }

    cout << "Total Students Present: " << count << endl;
}

// Main
int main()
{
    Student* head = NULL;

    int choice;

    do
    {
        cout << "\n===== University Attendance System =====\n";
        cout << "1. Add Student\n";
        cout << "2. Search Student\n";
        cout << "3. Delete Student\n";
        cout << "4. Display All Students\n";
        cout << "5. Count Students Present\n";
        cout << "6. Display Final Attendance List\n";
        cout << "7. Exit\n";

        cout << "Enter choice: ";

        cin >> choice;
    }

```

```
switch (choice)
{
    case 1:
        addStudent(head);
        break;

    case 2:
        searchStudent(head);
        break;

    case 3:
        deleteStudent(head);
        break;

    case 4:
        displayStudents(head);
        break;

    case 5:
        countPresent(head);
        break;

    case 6:
        displayStudents(head);
        break;
```

```

        case 7:

            cout << "Program ended.\n";

            break;

        default:

            cout << "Invalid choice.\n";

    }

} while (choice != 7);

return 0;

}

```

OUTPUT:

```

--- Attendance List ---
Roll No: 1 | Name: Usman Tariq | Status: Present
Roll No: 2 | Name: Ayesha Malik | Status: Present
Roll No: 3 | Name: Hamza Bilal | Status: Absent
Roll No: 4 | Name: Zainab Fatima | Status: Present

--- Search Operations ---
Student Found - Roll: 2 | Name: Ayesha Malik | Status: Present
Student not found.

--- Count Present ---
Total Students Present: 3

--- Delete Operation ---
Student with Roll Number 3 removed.

--- Final Attendance List ---
Roll No: 1 | Name: Usman Tariq | Status: Present
Roll No: 2 | Name: Ayesha Malik | Status: Present
Roll No: 4 | Name: Zainab Fatima | Status: Present

```

HOME TASK 03:

CODE:

```
#include <iostream>
```

```
#include <string>
```

```
using namespace std;
```

```
// Node
```

```
struct Order
```

```
{
```

```
    string orderID;
```

```
    string customerName;
```

```
    string foodItem;
```

```
    Order* next;
```

```
};
```

```
// Add order at end
```

```
void addOrder(Order*& head)
```

```
{
```

```
    Order* newOrder = new Order;
```

```
    cout << "Enter Order ID: ";
```

```
    cin >> newOrder->orderID;
```

```
    cout << "Enter Customer Name: ";
```

```
cin.ignore();  
getline(cin, newOrder->customerName);  
  
cout << "Enter Food Item: ";  
getline(cin, newOrder->foodItem);  
  
newOrder->next = NULL;  
  
if (head == NULL)  
{  
    head = newOrder;  
}  
else  
{  
    Order* temp = head;  
  
    while (temp->next != NULL)  
    {  
        temp = temp->next;  
    }  
  
    temp->next = newOrder;  
}  
  
cout << "Order added successfully.\n";  
}
```

```

// Add urgent order at beginning
void addUrgentOrder(Order*& head)
{
    Order* newOrder = new Order;

    cout << "Enter Urgent Order ID: ";
    cin >> newOrder->orderID;

    cout << "Enter Customer Name: ";
    cin.ignore();
    getline(cin, newOrder->customerName);

    cout << "Enter Food Item: ";
    getline(cin, newOrder->foodItem);

    newOrder->next = head;
    head = newOrder;

    cout << "Urgent order added at beginning.\n";
}

// Display orders
void displayOrders(Order* head)
{
    if (head == NULL)
    {
        cout << "No pending orders.\n";
    }
}

```

```
        return;
    }

    cout << "\nPending Orders:\n";

    Order* temp = head;

    while (temp != NULL)
    {
        cout << temp->orderID;

        if (temp->next != NULL)
        {
            cout << " -> ";
        }

        temp = temp->next;
    }

    cout << endl;
}

// Search order
void searchOrder(Order* head)
{
    string id;
```

```
cout << "Enter Order ID to search: ";
```

```
cin >> id;
```

```
Order* temp = head;
```

```
while (temp != NULL)
```

```
{
```

```
    if (temp->orderID == id)
```

```
    {
```

```
        cout << "\nOrder Found!\n";
```

```
        cout << "Order ID: " << temp->orderID << endl;
```

```
        cout << "Customer: " << temp->customerName << endl;
```

```
        cout << "Food Item: " << temp->foodItem << endl;
```

```
        return;
```

```
    }
```

```
    temp = temp->next;
```

```
}
```

```
cout << "Order not found.\n";
```

```
}
```

```
// Remove delivered order
```

```
void removeOrder(Order*& head)
```

```
{
```

```
    string id;
```

```
cout << "Enter Order ID to remove: ";
```

```
cin >> id;
```

```
if (head == NULL)
```

```
{
```

```
    cout << "Order not found.\n";
```

```
    return;
```

```
}
```

```
if (head->orderID == id)
```

```
{
```

```
    Order* temp = head;
```

```
    head = head->next;
```

```
    delete temp;
```

```
    cout << "Order delivered and removed.\n";
```

```
    return;
```

```
}
```

```
Order* temp = head;
```

```
while (temp->next != NULL)
```

```
{
```

```
    if (temp->next->orderID == id)
```

```
    {
```

```
        Order* deleteNode = temp->next;
```

```
temp->next = temp->next->next;
```

```
delete deleteNode;
```

```
cout << "Order delivered and removed.\n";
```

```
return;
```

```
}
```

```
temp = temp->next;
```

```
}
```

```
cout << "Order not found.\n";
```

```
}
```

```
// Main
```

```
int main()
```

```
{
```

```
Order* head = NULL;
```

```
int choice;
```

```
do
```

```
{
```

```
cout << "\n===== Food Delivery System =====\n";
```

```
cout << "1. Add New Order\n";
```

```
cout << "2. Display Pending Orders\n";
```

```
cout << "3. Search Order\n";  
cout << "4. Remove Delivered Order\n";  
cout << "5. Add Urgent Order\n";  
cout << "6. Display Updated Orders\n";  
cout << "7. Exit\n";
```

```
cout << "Enter choice: ";  
cin >> choice;
```

```
switch (choice)  
{  
    case 1:  
        addOrder(head);  
        break;  
  
    case 2:  
        displayOrders(head);  
        break;  
  
    case 3:  
        searchOrder(head);  
        break;  
  
    case 4:  
        removeOrder(head);  
        break;
```

case 5:

addUrgentOrder(head);

break;

case 6:

displayOrders(head);

break;

case 7:

cout << "Program ended.\n";

break;

default:

cout << "Invalid choice.\n";

}

} while (choice != 7);

return 0;

}

OUTPUT:

```
Pending Orders:
0101 -> 0102 -> 0103

Urgent Order 0104 received.
0104 -> 0101 -> 0102 -> 0103

Order 0101 delivered.
Order 0101 delivered successfully.
0104 -> 0102 -> 0103
```

HOME TASK 04:

CODE:

```
#include #include using namespace std;

// Node struct Course { string code; string name; int creditHours; Course* next; };

// Add course at beginning void addAtBeginning(Course*& head) { Course* newCourse = new
Course;

cout << "Enter Course Code: ";
cin >> newCourse->code;

cout << "Enter Course Name: ";
cin.ignore();
getline(cin, newCourse->name);

cout << "Enter Credit Hours: ";
cin >> newCourse->creditHours;

newCourse->next = head;
head = newCourse;

cout << "Course added at beginning.\n";

}

// Add course at end void addAtEnd(Course*& head) { Course* newCourse = new Course;
```

```

cout << "Enter Course Code: ";
cin >> newCourse->code;

cout << "Enter Course Name: ";
cin.ignore();
getline(cin, newCourse->name);

cout << "Enter Credit Hours: ";
cin >> newCourse->creditHours;

newCourse->next = NULL;

if (head == NULL)
{
    head = newCourse;
}
else
{
    Course* temp = head;

    while (temp->next != NULL)
    {
        temp = temp->next;
    }

    temp->next = newCourse;
}

cout << "Course added at end.\n";

}

// Search course void searchCourse(Course* head) { string code;

cout << "Enter Course Code to search: ";
cin >> code;

Course* temp = head;

while (temp != NULL)
{

```

```

    if (temp->code == code)
    {
        cout << "\nCourse Found!\n";
        cout << "Course Code: " << temp->code << endl;
        cout << "Course Name: " << temp->name << endl;
        cout << "Credit Hours: " << temp->creditHours << endl;

        return;
    }

    temp = temp->next;
}

cout << "Course not found.\n";

}

// Delete course void deleteCourse(Course*& head){ string code;

cout << "Enter Course Code to delete: ";
cin >> code;

if (head == NULL)
{
    cout << "Course not found.\n";
    return;
}

if (head->code == code)
{
    Course* temp = head;
    head = head->next;

    delete temp;

    cout << "Course deleted successfully.\n";
    return;
}

Course* temp = head;

```

```

while (temp->next != NULL)
{
    if (temp->next->code == code)
    {
        Course* deleteNode = temp->next;

        temp->next = temp->next->next;

        delete deleteNode;

        cout << "Course deleted successfully.\n";
        return;
    }

    temp = temp->next;
}

cout << "Course not found.\n";

}

// Display courses void displayCourses(Course* head){ if (head == NULL){ cout << "Course list is
empty.\n"; return; }

Course* temp = head;

cout << "\nCourses:\n";

while (temp != NULL)
{
    cout << temp->code;

    if (temp->next != NULL)
    {
        cout << " -> ";
    }

    temp = temp->next;
}

```

```

cout << endl;

}

// Count courses void countCourses(Course* head) { int count = 0;

Course* temp = head;

while (temp != NULL)
{
    count++;
    temp = temp->next;
}

cout << "Total Courses: " << count << endl;

}

// Concatenate two course lists void concatenate(Course*& list1, Course* list2) { if (list1 == NULL)
{ list1 = list2; return; }

Course* temp = list1;

while (temp->next != NULL)
{
    temp = temp->next;
}

temp->next = list2;

}

// Create sample course void addSampleCourse(Course*& head, string code, string name, int
credits) { Course* newCourse = new Course;

newCourse->code = code;
newCourse->name = name;
newCourse->creditHours = credits;
newCourse->next = NULL;

```

```

if (head == NULL)
{
    head = newCourse;
}
else
{
    Course* temp = head;

    while (temp->next != NULL)
    {
        temp = temp->next;
    }

    temp->next = newCourse;
}

}

// Main
int main() { Course* morning = NULL; Course* evening = NULL;

int choice;
int listChoice;

do
{
    cout << "\n===== University Course Management =====\n";
    cout << "1. Add Course at Beginning\n";
    cout << "2. Add Course at End\n";
    cout << "3. Search Course\n";
    cout << "4. Delete Course\n";
    cout << "5. Display All Courses\n";
    cout << "6. Count Total Courses\n";
    cout << "7. Concatenate Another Course List\n";
    cout << "8. Exit\n";

    cout << "Enter choice: ";
    cin >> choice;

    if (choice >= 1 && choice <= 6)
    {
        cout << "\nSelect Course List:\n";
        cout << "1. Morning Courses\n";
    }
}

```

```

    cout << "2. Evening Courses\n";
    cout << "Enter choice: ";
    cin >> listChoice;
}

switch (choice)
{
    case 1:

        if (listChoice == 1)
            addAtBeginning(morning);
        else if (listChoice == 2)
            addAtBeginning(evening);
        else
            cout << "Invalid list.\n";

        break;

    case 2:

        if (listChoice == 1)
            addAtEnd(morning);
        else if (listChoice == 2)
            addAtEnd(evening);
        else
            cout << "Invalid list.\n";

        break;

    case 3:

        if (listChoice == 1)
            searchCourse(morning);
        else if (listChoice == 2)
            searchCourse(evening);
        else
            cout << "Invalid list.\n";

        break;

    case 4:

```

```
    if (listChoice == 1)
        deleteCourse(morning);
    else if (listChoice == 2)
        deleteCourse(evening);
    else
        cout << "Invalid list.\n";

    break;
```

case 5:

```
    if (listChoice == 1)
        displayCourses(morning);
    else if (listChoice == 2)
        displayCourses(evening);
    else
        cout << "Invalid list.\n";

    break;
```

case 6:

```
    if (listChoice == 1)
        countCourses(morning);
    else if (listChoice == 2)
        countCourses(evening);
    else
        cout << "Invalid list.\n";

    break;
```

case 7:

```
{
    cout << "\nWhich list should be joined with the other?\n";
    cout << "Morning + Evening\n";

    concatenate(morning, evening);

    cout << "\nCombined Course List:\n";
    displayCourses(morning);
```

```

        break;
    }

    case 8:
        cout << "Program ended.\n";
        break;

    default:
        cout << "Invalid choice.\n";
    }

} while (choice != 8);

return 0;

}

```

OUTPUT:

```

Morning Courses:
CS101 -> AI201 -> DS301

Evening Courses:
SE101 -> WEB201 -> DB301

--- Search Operation ---
Course Found: [AI201] Artificial Intelligence (3 Credit Hours)
Course with code MATH101 not found.

--- Concatenating Evening Courses into Morning Courses ---
Combined Course List:
CS101 -> AI201 -> DS301 -> SE101 -> WEB201 -> DB301

Total Courses: 6

```